

第12課

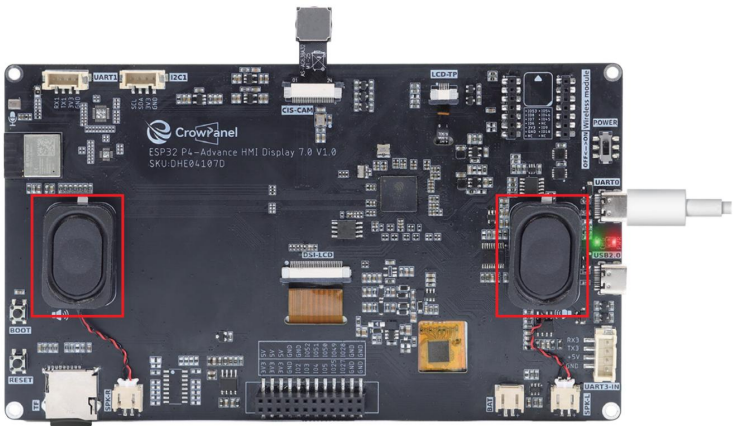
SDカードからローカル音楽を再生

導入

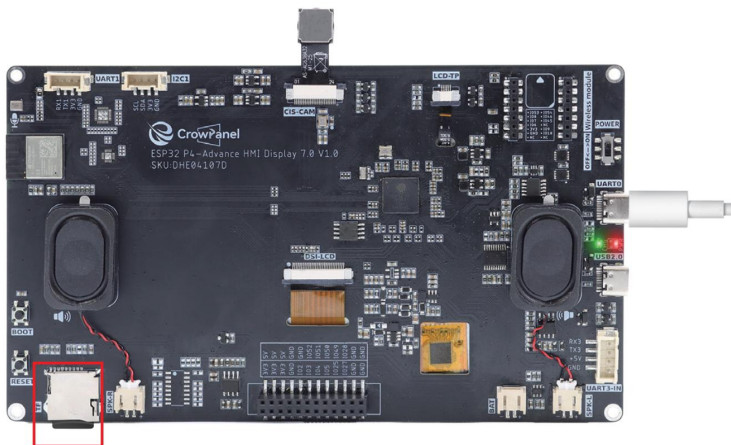
このレッスンでは、以前のレッスンで使用したbsp_sdコンポーネントとbsp_audioコンポーネントを使用して、SDカードに保存されているWAVオーディオファイルを再生します。

このレッスンで使用するハードウェア

Advance-P4のスピーカー

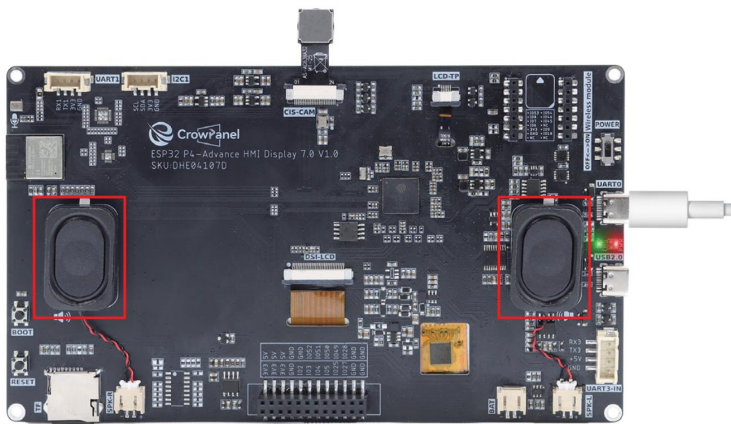


Advance-P4にSDカードを挿入



動作効果図

コードを実行すると、SDカードに保存されているWAV形式の音声ファイルがAdvance-P4のスピーカーから再生されます。



SDカードに保存されているWAV音声ファイルが再生されています。

重要な説明

- このレッスンの主な焦点は、`bsp_sd` と `bsp_audio` という 2 つのコンポーネントを組み合わせ使用することです。実際、SD カード コンポーネントでは、前のコンポーネントと同じインターフェースを使用します。これらのインターフェースについては既に詳しく説明したので、ここでは改めて説明しません。

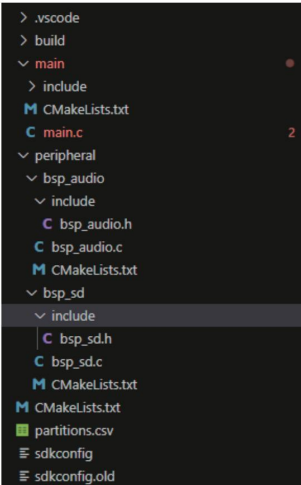
- 次に、`bsp_audio` コンポーネントの理解に焦点を当てます。このコンポーネントは、前回のレッスンで録音後 5 秒経過してから元の音声再生するために使用されました。当時、私たちはすでにこのコンポーネントについてある程度の知識を得ていましたが、スピーカーをオンにする方法しか学びませんでした。今回のレッスンでは、難易度を少し上げて、WAV 形式の音声を再生する方法を学びます。

- まず、下記のGitHubリンクをクリックして、このレッスンのコードをダウンロードしてください。

https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson12-Playing_Loca_Music_from_SD_Card

- 次に、このレッスンのコードをVS Codeにドラッグして、プロジェクトファイルを開きます。

- 開くと、このプロジェクトの枠組みが表示されます。



このレッスンの例では、`peripheral\` ディレクトリの下に `bsp_sd` と `bsp_audio` という名前の新しいフォルダが作成されます。

`bsp_audio` フォルダ内に、新しい `include` フォルダと「`CMakeLists.txt`」ファイルが作成されます。（`bsp_sd` にも同じ構造が適用されます。）

`bsp_audio` フォルダには「`bsp_audio.c`」ドライバファイルが含まれており、`include` フォルダには「`bsp_audio.h`」ヘッダーファイルが含まれています。（`bsp_sd` にも同じファイル構造が適用されます。）

「`CMakeLists.txt`」ファイルは、ドライバをビルドシステムに統合します。これにより、プロジェクトは「`bsp_audio.c`」で定義されている関数（WAV オーディオの解析やSDカードからのWAVオーディオの再生など）と、「`bsp_sd.c`」で定義されている関数（SDカードの初期化やSDカード情報の取得など）を利用できるようになります。

bsp_audio コード

まず、音声再生コンポーネントを見てみましょう。これには「`bsp_audio.c`」と「`bsp_audio.h`」という2つのファイルが含まれています。

- 次に、まず「bsp_audio.h」プログラムを解析します。
- "bsp_audio.h" はオーディオ再生モジュールのヘッダーファイルで、主に以下の目的で使用されます。
- 外部プログラム向けに「bsp_audio.c」で実装されている関数、マクロ、変数を宣言することで、他の .c ファイルから「#include "bsp_audio.h"」だけでこのモジュールを呼び出すことができます。
- 言い換えれば、モジュールの内部の詳細を隠蔽しつつ、外部に利用可能な関数や定数を通知するインターフェース層です。
- このコンポーネントでは、使用する必要のあるすべてのライブラリが「bsp_audio.h」ファイルに配置され、一元管理されています。

```

4  /*-----Header file declaration-----*/
5  #include "esp_log.h" //References for LOG Printing Function-related API Functions
6  #include "esp_err.h" //References for Error Type Function-related API Functions
7  #include "driver/gpio.h" //References for GPIO Function-related API Functions
8  #include "driver/i2s_std.h" //References for I2S STD Function-related API Functions
9  #include "bsp_sd.h"

```

- 次に、使用する必要のある変数と関数を宣言します。これらの関数の具体的な実装は「bsp_audio.c」にあります。
- これらを「bsp_audio.h」に統一的に配置したのは、呼び出しと管理の利便性を高めるためです。（これらの関数の機能については、「bsp_audio.c」で使用したときに説明します。）

```

15 /*-----Variable declaration-----*/
16 #define AUDIO_TAG "AUDIO"
17 #define AUDIO_INFO(fmt, ...) ESP_LOGI(AUDIO_TAG, fmt, ##_VA_ARGS_)
18 #define AUDIO_DEBUG(fmt, ...) ESP_LOGD(AUDIO_TAG, fmt, ##_VA_ARGS_)
19 #define AUDIO_ERROR(fmt, ...) ESP_LOGE(AUDIO_TAG, fmt, ##_VA_ARGS_)
20
21 #define AUDIO_GPIO_LRCLK 21
22 #define AUDIO_GPIO_BCLK 22
23 #define AUDIO_GPIO_SDATA 23
24 #define AUDIO_GPIO_CTRL 30
25
26 esp_err_t audio_init();
27 esp_err_t audio_ctrl_init();
28 esp_err_t set_audio_ctrl(bool state);
29 i2s_chan_handle_t get_audio_handle();

```

- それでは、「bsp_audio.c」内の各関数の具体的な機能を見ていきましょう。
- bsp_audio.h: このプロジェクト用のカスタムオーディオモジュールヘッダーファイルで、以下を定義します
マクロ、GPIOピン、および関数宣言。

```

1  /*-----Header file declaration-----*/
2  #include "bsp_audio.h"
3  /*-----Header file declaration end-----*/

```

- グローバル変数 tx_chan が定義され、その型は i2s_chan_handle_t、つまり I2S です。
チャンネルハンドル。
- このハンドルは音声出力チャンネル（TX）を表し、以降のすべての音声再生操作はこのチャンネルを通して実行されます。

```

5  /*-----Variable declaration-----*/
6  i2s_chan_handle_t tx_chan;
7  /*-----Variable declaration end-----*/

```

audio_init()

この関数は、ESP32のI2Sオーディオ出力チャンネルを初期化し、16kHz、16ビット、ステレオ形式でオーディオを再生できるようにするために使用されます。I2S伝送チャンネルを作成し、標準的なオーディオパラメータ（サンプリングレート、ビット幅、左右チャンネル、GPIOピンなど）を設定して、チャンネルを開始し、オーディオ出力の準備を行います。

- `esp_err_t err = ESP_OK;` エラー状態変数を初期化します。デフォルト値は
手続は成功しました。
- `i2s_chan_config_t chan_cfg = {...};` I2S伝送チャンネルを設定します
パラメータ:
- ID: I2Sコントローラー1を使用
- 役割: マスターモード（クロック信号を生成する）
- `dma_desc_num` および `dma_frame_num`: DMA バッファサイズ設定
- `auto_clear`: DMAバッファのアンダーフローを自動的にクリアします
- `intr_priority`: 割り込み優先度
- `i2s_new_channel(&chan_cfg, &tx_chan, NULL);` 新しいI2S送信を作成します
チャンネルを呼び出し、`tx_chan`に保存します。
- `i2s_std_config_t std_cfg = {...};` 標準I2Sオーディオパラメータを設定します。
- `clk_cfg`: クロック設定（サンプリングレート16kHz、マスタークロック乗数256）
- `slot_cfg`: 音声データ形式（16ビット、ステレオ、左揃え）
- `gpio_cfg`: I2S信号（BCLK、LRCLK、SDATA出力）に対応するGPIOピンと
それらを反転させるかどうか
- `i2s_channel_init_std_mode(tx_chan, &std_cfg);` I2S伝送チャンネルを標準モードで初期化し、チャンネルが上記のクロック、データフォーマット、およびGPIO構成に準拠するようにします。
- `i2s_channel_enable(tx_chan);` I2Sチャンネルの動作を開始し、
音声データを送信する。
- `return err;` 初期化ステータスを返します。途中でエラーが発生した場合は、エラーコードを返します。
事前に返却されます。

この機能の主な目的は、I2Sオーディオ伝送チャンネルを作成および構成し、ESP32-P4が指定されたGPIOを介して16kHz、16ビット、ステレオ形式のオーディオを出力できるようにすることです。

```
peripheral > bsp_audio > C bsp_audio.c > G audio_init()
12  esp_err_t audio_init()
13
14      esp_err_t err = ESP_OK;
15      i2s_chan_config_t chan_cfg = {
16          .id = I2S_NUM_1,           /*Use I2S controller 1*/
17          .role = I2S_ROLE_MASTER,   /*I2S acts as master (generates clock signals)*/
18          .dma_desc_num = 6,         /*Number of DMA descriptors for buffer management*/
19          .dma_frame_num = 256,      /*Number of frames per DMA descriptor*/
20          .auto_clear = true,        /*Automatically clear DMA buffer on underflow*/
21          .intr_priority = 0,        /*Interrupt priority level*/
22      }; /*I2S channel configuration for transmitter (audio output)*/
23      err = i2s_new_channel(&chan_cfg, &tx_chan, NULL); /*Create new I2S channel (transmit channel only, no receive channel)*/
24      if (err != ESP_OK)
25          return err;
26      i2s_std_config_t std_cfg = {
27          .clk_cfg = {
28              .sample_rate_hz = 16000, /*Audio sample rate: 16kHz*/
29              .clk_src = I2S_CLK_SRC_DEFAULT, /*Default clock source*/
30              .mclk_multiple = I2S_MCLK_MULTIPLE_256, /*Master clock multiplier*/
31          },
32          .slot_cfg = {
33              .data_bit_width = I2S_DATA_BIT_WIDTH_16BIT, /*16-bit audio samples*/
34              .slot_bit_width = I2S_SLOT_BIT_WIDTH_AUTO, /*Auto-calculate slot width*/
35              .slot_mode = I2S_SLOT_MODE_STEREO, /*Stereo audio (2 channels)*/
36              .slot_mask = I2S_STD_SLOT_BOTH, /*Enable both left and right channels*/
37              .ws_width = I2S_DATA_BIT_WIDTH_16BIT, /*Word select signal width*/
38              .ws_pol = false, /*Word select polarity (normal)*/
39              .bit_shift = true, /*Enable bit shift in data frame*/
40              .left_align = true, /*Left-aligned data in slot*/
41              .big_endian = false, /*Little-endian byte order*/
42              .bit_order_lsb = false, /*MSB first bit order*/
43          },
44          .gpio_cfg = {
45              .mclk = I2S_GPIO_UNUSED, /*Master clock not used*/
46              .bclk = AUDIO_GPIO_BCLK, /*Bit clock pin*/
47              .ws = AUDIO_GPIO_LRCLK, /*Word select (left/right clock) pin*/
48              .dout = AUDIO_GPIO_SDATA, /*Serial data output pin*/
49              .din = I2S_GPIO_UNUSED, /*Data input not used (output only)*/
50              .invert_flags = {
51                  .mclk_inv = false, /*Don't invert master clock*/
52                  .bclk_inv = false, /*Don't invert bit clock*/
53                  .ws_inv = false, /*Don't invert word select*/
54              },
55              .*, /*Signal inversion flags*/
56          }, /*GPIO pin configuration for I2S signals*/
57      }; /*Standard I2S configuration for audio playback*/
58      err = i2s_channel_init_std_mode(tx_chan, &std_cfg); /*Initialize I2S channel in standard mode for audio output*/
59      if (err != ESP_OK)
60          return err;
```

したがって、後で使用するオーディオファイルはすべて、この要件（16kHzのサンプリングレート、16ビットのビット深度、ステレオ形式、つまりデュアルチャンネル）を満たす必要があります。

```
i2s_std_config_t std_cfg = {
    .clk_cfg = {
        .sample_rate_hz = 16000, /*Audio sample rate: 16kHz*/
        .clk_src = I2S_CLK_SRC_DEFAULT, /*Default clock source*/
        .mclk_multiple = I2S_MCLK_MULTIPLE_256, /*Master clock multiplier*/
    },
    .slot_cfg = {
        .data_bit_width = I2S_DATA_BIT_WIDTH_16BIT, /*16-bit audio samples*/
        .slot_bit_width = I2S_SLOT_BIT_WIDTH_AUTO, /*Auto-calculate slot width*/
        .slot_mode = I2S_SLOT_MODE_STEREO, /*Stereo audio (2 channels)*/
        .slot_mask = I2S_STD_SLOT_BOTH, /*Enable both left and right channels*/
        .ws_width = I2S_DATA_BIT_WIDTH_16BIT, /*Word select signal width*/
        .ws_pol = false, /*Word select polarity (normal)*/
        .bit_shift = true, /*Enable bit shift in data frame*/
        .left_align = true, /*Left-aligned data in slot*/
        .big_endian = false, /*Little-endian byte order*/
        .bit_order_lsb = false, /*MSB first bit order*/
    },
    /* Audio slot/data format configuration*/
};
```

`audio_ctrl_init:`

この関数は、オーディオパワーアンプの制御ピンを初期化し、出力モードとして構成することで、後続のパワーアンプのオン/オフ状態を制御するために使用されます。

```

66  esp_err_t audio_ctrl_init()
67  {
68      esp_err_t err = ESP_OK;
69      const gpio_config_t audio_gpio_cofig = {
70          .pin_bit_mask = 1ULL << AUDIO_GPIO_CTRL,
71          .mode = GPIO_MODE_OUTPUT,
72          .pull_up_en = false,
73          .pull_down_en = false,
74          .intr_type = GPIO_INTR_DISABLE,
75      };
76      err = gpio_config(&audio_gpio_cofig);
77      if (err != ESP_OK)
78          return err;
79      return err;

```

`set_Audio_ctrl:`

この機能は、オーディオパワーアンプのオン/オフ状態を制御するために使用されます。パワーアンプ制御ピンのレベル（アクティブロー）を設定することで、パワーアンプのオン/オフを切り替えます。

```

82  esp_err_t set_Audio_ctrl(bool state)
83  {
84      esp_err_t err = ESP_OK;
85      bool status = !state;
86      err = gpio_set_level(AUDIO_GPIO_CTRL, status);
87      return err;
88  }

```

`validate_wav_header():`

この関数は、開いたWAVファイルのヘッダーが有効かどうか、ファイルが標準のPCM WAV形式であるかどうか、一般的なサンプリングレート、チャンネル数、ビット深度に対応しているかどうかをチェックするために使用されます。検証後、この関数はファイルの読み取り状態を変更せずに、ファイルポインタを元の位置に戻します。

- `if (file == NULL)` ファイルポインタが null かどうかをチェックします。null の場合は false を返します。
- `long original_position = ftell(file);` ファイルポインタの現在の位置を取得します
その後の修復。
- `if (original_position == -1)` ファイル位置が正常に取得されたかどうかを確認します。
- `fseek(file, 0, SEEK_SET)` ファイルポインタをファイルの先頭に移動します。
- `uint8_t header[44]; size_t bytes_read = fread(header, 1, 44, file);` 最初の44バイトを読み込む
WAVファイルのバイト列（標準WAVファイルヘッダー）。

- `if (bytes_read != 44)` WAV ヘッダーが完全に読み込まれたかどうかを確認します。
- `memcmp(header, "RIFF", 4)` ファイルが「RIFF」(RIFF チャンク)で始まるかどうかを確認します (識別子)。
- `memcmp(header + 8, "WAVE", 4)` フォーマットが「WAVE」かどうかを確認します。
- `memcmp(header + 12, "fmt ", 4)` `fmt` サブチャンクの存在を確認します。
- `uint16_t audio_format = *(uint16_t*)(header + 20);` オーディオフォーマットを取得します (フィールド (1はPCMを示す))。
- `uint16_t num_channels = *(uint16_t*)(header + 22);` チャンネル数を取得します (1 または 2 チャンネルをサポートします)。
- `uint32_t sample_rate = *(uint32_t*)(header + 24);` サンプリングレートを取得します (そして、それが一般的に使用されている値であるかどうかを確認します)。
- `uint16_t bits_per_sample = *(uint16_t*)(header + 34);` サンプルあたりのビット数を取得します (8/16/24/32 ビットをサポート)。
- `memcmp(header + 36, "data", 4)` データチャンク識別子が「data」であるかどうかを確認します。
- `uint32_t file_size = *(uint32_t*)(header + 4) + 8; uint32_t data_size = *(uint32_t*)(header + 40);` 印刷用に合計ファイルサイズとオーディオデータサイズを取得します (情報)。
- `AUDIO_INFO(...)` WAV ファイル情報 (チャンネル数、サンプリングレート、ビット深度、データサイズ、ファイルサイズ)。
- `fseek(file, original_position, SEEK_SET);` ファイルポインタを元の位置に戻します (位置)。
- `return true;` 検証が成功した場合は `true` を返します。

この関数の役割は、WAVファイルヘッダーの有効性をチェックし、ファイルが標準的なPCM WAV形式であること、一般的なサンプリングレート、ビット深度、チャンネル数をサポートしていることを確認し、検証後にファイルポインタの位置を復元することです。

- 最初の44バイトは標準的なPCM WAVヘッダーを構成し、オーディオフォーマット、チャンネル数、サンプリングレートなどの情報を記述します。
- WAVファイルを再生または処理する前に、通常は読み込みと検証が必要です。
このヘッダーは、ファイル形式が期待どおりであることを保証するために使用されます。
- `validate_wav_header()` 関数は、以下の基準に従って各フィールドの有効性をチェックします。
構造。

オーディオ再生_wav_sd:

Audio_play_wav_sd() は、SD カードから WAV ファイルを読み込み、ESP32 の I2S 出力を介してオーディオを再生するために使用されます。WAV ファイルのヘッダーを検証し、ヘッダーをスキップし、オーディオデータをチャンク単位で読み込み、音量を処理し（増幅と範囲制限）、オーディオ再生が完了するまでデータを I2S プレーヤーに送信し、その後解放します。

リソース。

- esp_err_t err = ESP_OK; エラー状態変数を初期化します。
- if (filename == NULL) 入力ファイル名がNULLかどうかをチェックし、パラメータエラーを返します
もしそうなら、
- FILE *fh = fopen(filename, "rb"); WAV ファイルを読み取り専用のバイナリ モードで開きます。
- if (fh == NULL) ファイルのオープンに失敗した場合にエラーを返します。
- if (!validate_wav_header(fh)) 以前に記述した WAV ヘッダー検証を呼び出す
フォーマットが正しいかどうかを確認する関数。
- fseek(fh, 44, SEEK_SET) WAVファイルのヘッダー(44バイト)をスキップして読み取りの準備をします
音声データ。
- バッファサイズを定義する
 - SAMPLES_PER_BUFFER = 512 1回に読み取られるサンプル数
 - INPUT_BUFFER_SIZE, OUTPUT_BUFFER_SIZE 入力バッファと出力バッファのバイトサイズ
- heap_caps_malloc(...) SPI RAM に入力バッファと出力バッファを割り当てます。割り当てが成功した場合
失敗すると、割り当てられたリソースを解放して終了します。
- 読み取りと書き込み用の変数を初期化します: samples_read, bytes_to_write,
書き込みバイト数、総サンプル数、ボリュームデータ。
- set_Audio_ctrl(true); オーディオハードウェアまたはアンプをオンにします。
- while (1) 音声データを読み込んで再生するループ:
- samples_read = fread(...) ファイルからオーディオサンプルを読み込み、入力バッファに格納します
- if (samples_read == 0) break; ファイル読み込みが完了したらループを終了します
- for ループ モノラルサンプルを 10 倍に増幅し、int16 の範囲に制限し、
それらを出力バッファに格納します（ここでは左チャンネルに使用できますが、ステレオに拡張することもできます）。
- bytes_to_write = samples_read * sizeof(int16_t); バイト数を計算します
I2Sに書き込まれる

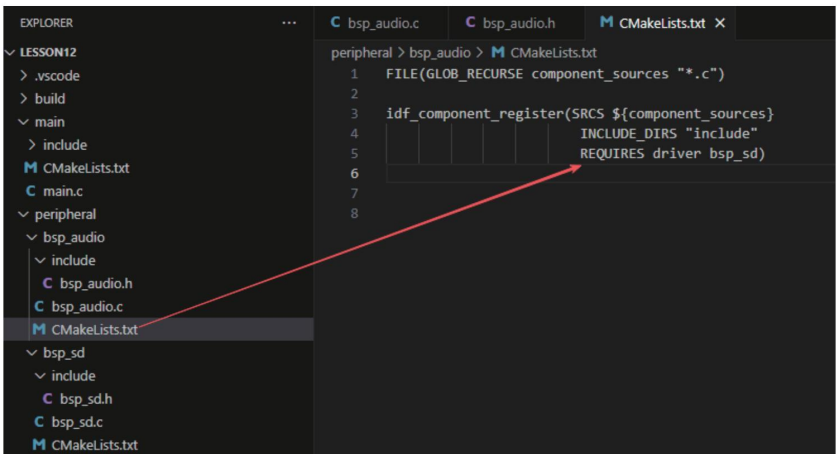
- `i2s_channel_write(tx_chan, output_buf, ...)` オーディオデータをI2S出力に書き込みます
チャンネル
- エラーチェック: 書き込みが失敗した場合はエラーを表示してループを終了します
- 再生されたサンプルの総数をカウントするために、`total_samples` を累積します。
- ループ終了後にリソースをクリーンアップします。
 - `set_Audio_ctrl(false);` オーディオハードウェアをオフにします
 - `free(input_buf); free(output_buf); fclose(fh);` バッファを解放し、ファイルを閉じます
- `AUDIO_INFO(...)` 再生完了情報を表示します
- `return err;` 再生結果ステータスを返します

この関数は、SDカードからWAVファイルを読み込み、フォーマットを検証した後、音声をチャック単位で再生し、I2Sを介してオーディオハードウェアに出力し、音量とバッファの管理を行い、再生後にすべてのリソースを解放します。

以上がbsp_audioコンポーネントの紹介です。これらのインターフェースの呼び出し方さえ知っていれば十分です。

これら呼び出すには、bsp_audioフォルダ内の「CMakeLists.txt」も設定する必要があります。

bsp_audioフォルダに配置されるこのファイルは、主にESP-IDFビルドシステム（CMake）に対して、bsp_audioコンポーネントをコンパイルおよび登録する方法を指示する役割を果たします。



- 「driver」と「bsp_sd」がここに含まれている理由は、「bsp_audio.h」でそれら呼び出しているためです（他のライブラリはシステムライブラリなので、追加する必要はありません）。

```

C bsp_audio.c  C bsp_audio.h 6 X  M CMakeLists.txt
peripheral > bsp_audio > include > C bsp_audio.h > AUDIO_GPIO_CTRL
1  #ifndef _BSP_AUDIO_H_
2  #define _BSP_AUDIO_H_
3
4  /*-----Header file declaration-----*/
5  #include "esp_log.h" //References for LOG Printing Function-related API Functions
6  #include "esp_err.h" //References for Error Type Function-related API Functions
7  #include "driver/gpio.h" //References for GPIO Function-related API Functions
8  #include "driver/i2s_std.h" //References for I2S STD Function-related API Functions
9  #include "bsp_sd.h"

```

- SDカード読み取り操作にはSDコンポーネントのインターフェースを使用します。
その他。
- bsp_sd コンポーネントについては、以前のレッスンで詳しく説明したので、
ここでは繰り返しません。このコンポーネントを直接使用します。

MP3をWAVに変換する

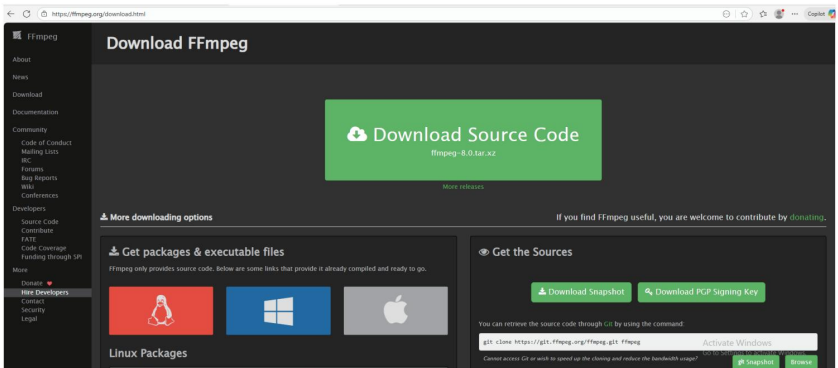
前述のとおり、このレッスンのコードに基づいて音声を再生する場合、音声ファイルは16kHzのサンプリングレート、16ビットのビット深度、ステレオ形式（つまりデュアルチャンネル）のWAVファイルである必要があります。

次に、MP3オーディオファイルを、16kHz、16ビット、ステレオ（デュアルチャンネル）の仕様を満たすWAVオーディオファイルに変換する方法をご紹介します。

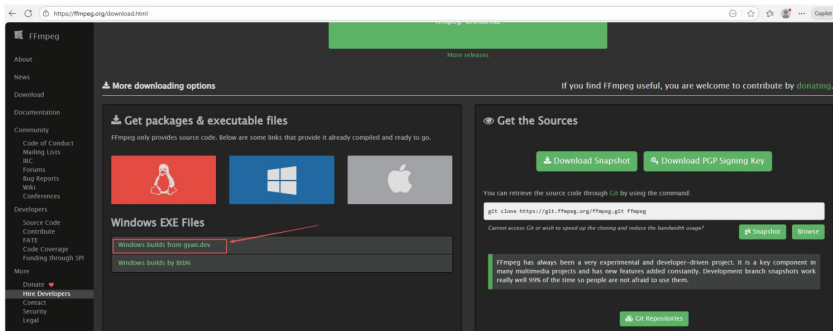
FFmpegは、ビデオやオーディオなどのマルチメディアファイルを処理するためのオープンソースのツールキットです。ほぼすべてのマルチメディア形式の変換、カット、編集をサポートしており、開発者やマルチメディア専門家にとって不可欠なツールとなっています。

FFmpegをダウンロードするには、以下のリンクを開いてください。

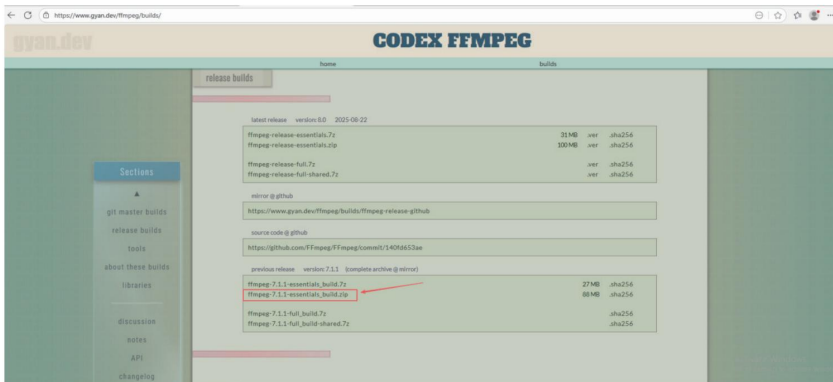
<https://ffmpeg.org/download.html>



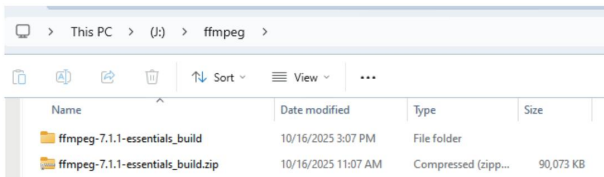
- Windowsを例にとると: 「Windowsがビルドする」インストールパッケージを選択します
gyan.dev。



- 下にスクロールして「リリースビルド」セクションを見つけ、次に
「ffmpeg-7.1.1-essentials_build.zip」

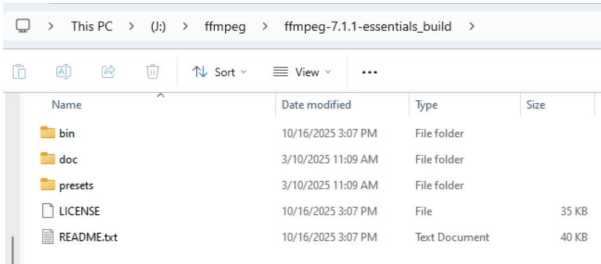


- ダウンロードが完了したら、ファイルを解凍して「Ffmpeg」フォルダを取得してください。



推奨される保存パス

フォルダを解凍して、システムドライブ（Cドライブ以外）に保存することをお勧めします。これにより、Cドライブ（システムドライブ）の容量を占有することと、システムの安定性とパフォーマンスを確保できます。

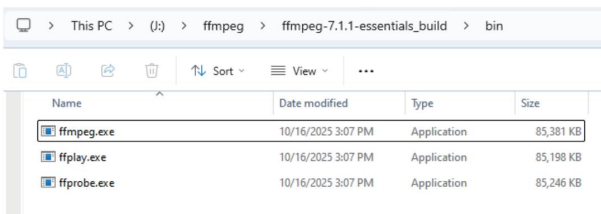


抽出されたフォルダのディレクトリ構造

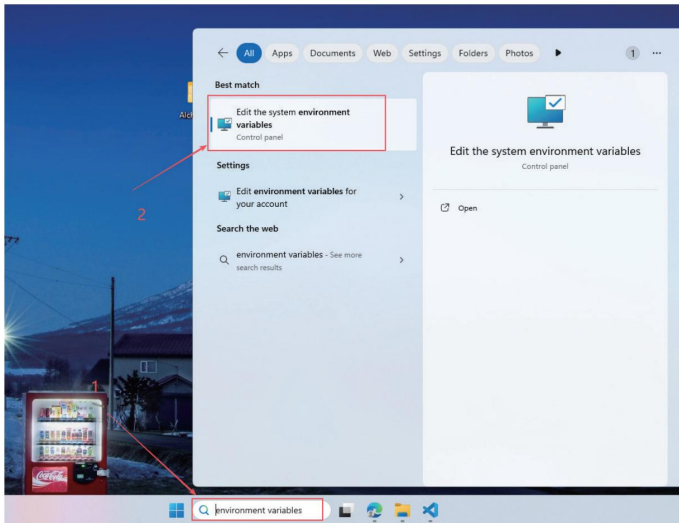
展開されたフォルダには、以下のディレクトリが含まれているはずです。

- "bin": FFmpeg実行可能ファイルを含むフォルダ。FFmpegを実行するためのすべてのコマンド。このディレクトリ内のファイルを介して実行する必要があります。
- "doc":ドキュメントおよび参考資料。
- 「プリセット」 :事前に設定されたフォーマットとエンコード方式。

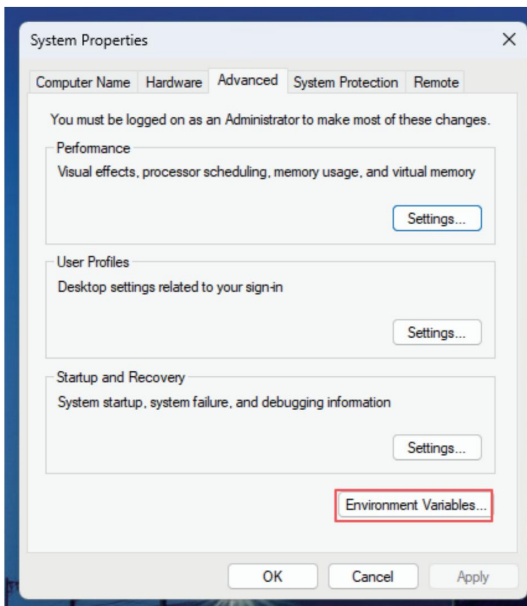
「bin」ディレクトリに移動すると、FFmpegのコア実行ファイルである「ffmpeg.exe」、「ffplay.exe」、 「ffprobe.exe」の3つが表示されます。



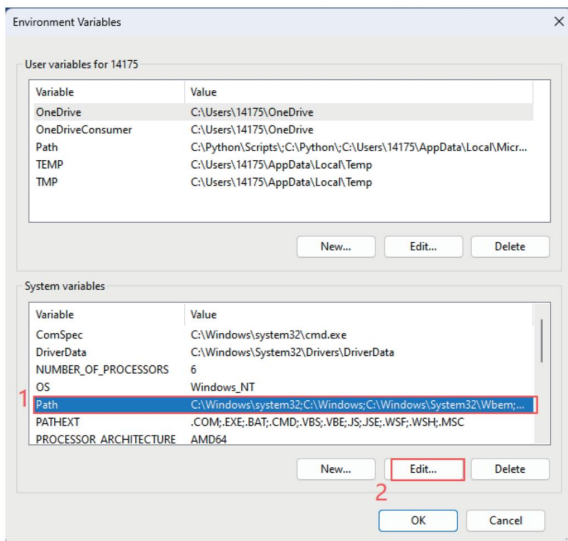
- コマンドラインで FFmpeg を直接呼び出すには、それを次の場所に追加する必要があります。
システムの環境変数。
- デスクトップ左下のスタートメニューで「環境変数」を検索し、「システム環境変数の編集」を見つけてクリックして開きます。



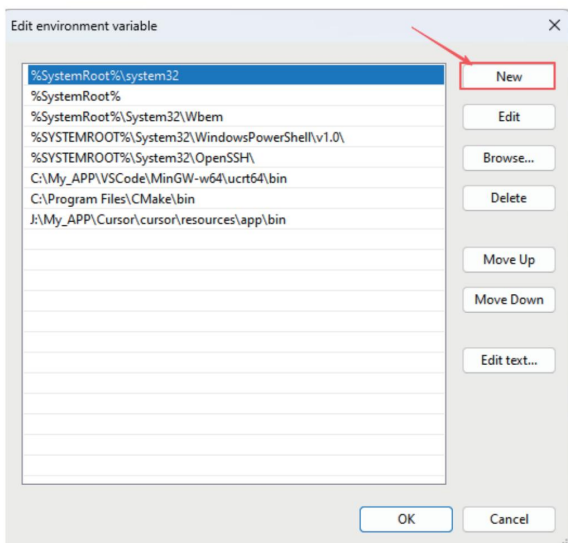
- 「環境変数」ボタンをクリックします。



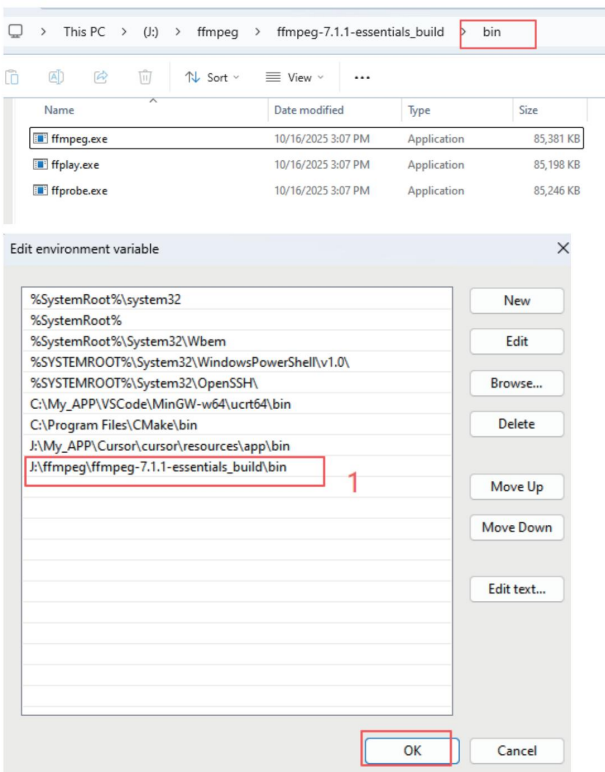
- 「システム変数」の下にある「パス」エントリを見つけて、「編集」をクリックします。



- 「環境変数の編集」ウィンドウで、「新規」をクリックします。



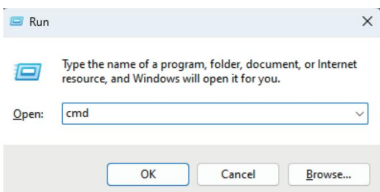
- FFmpegの「bin」フォルダへのパスを入力してください（ご自身のFFmpegパスを使用してください）



- パスを入力した後は、必ず設定を保存してください。

注：システムがFFmpegファイルを正しく検出できるように、パスが正確であることを確認してください。

- FFmpegのインストールが成功したことを確認する
- WindowsキーとRキーを同時に押して、「cmd」と入力すると、コマンドプロンプトウィンドウが開きます。



- Ffmpegのバージョンを確認するには、コマンドラインに次のコマンドを入力してください。
ffmpeg -version
- Ffmpegのバージョン番号と関連情報が正しく表示されれば、インストールは成功しています（下図参照）。

```

C:\Users\14175>ffmpeg -version
ffmpeg version 7.1.1-essentials_build-www.gyan.dev Copyright (c) 2000-2025 the FFmpeg developers
built with gcc 14.2.0 (Rev1, Built by MSYS2 project)
configuration: --enable-gpl --enable-version3 --enable-static --disable-w32threads --disable-autodetect --enable-fontcon
fig --enable-iconv --enable-gnutls --enable-libxml2 --enable-gmp --enable-bzlib --enable-lzma --enable-zlib --enable-lib
srt --enable-libssh --enable-libzmq --enable-avisynth --enable-sdl2 --enable-libwebp --enable-libx264 --enable-libx265 --
enable-libxvid --enable-libaom --enable-libopenjpeg --enable-libvpx --enable-mediafoundation --enable-libass --enable-l
ibfreetype --enable-libfribidi --enable-libharfbuzz --enable-libvidstab --enable-libvmaf --enable-libzimg --enable-amf --
enable-cuda-llvm --enable-cuvid --enable-dxva2 --enable-d3d11va --enable-d3d12va --enable-ffnvcodec --enable-libvpl --e
nable-nvdec --enable-nvenc --enable-vpapi --enable-libgsm --enable-libopenmpt --enable-libopencore-amrnb --enable-libomp3
lame --enable-libtheora --enable-libvo-amrwbenc --enable-libgsm --enable-libopencore-amrnb --enable-libopus --enable-lib
speex --enable-libvorbis --enable-librubberband
libavutil      59. 39.100 / 59. 39.100
libavcodec     61. 19.101 / 61. 19.101
libavformat    61.  7.100 / 61.  7.100
libavdevice    61.  3.100 / 61.  3.100
libavfilter    10.  4.100 / 10.  4.100
libswscale     8.  3.100 /  8.  3.100
libswresample  5.  3.100 /  5.  3.100
libpostproc   58.  3.100 / 58.  3.100

C:\Users\14175>

```

- 次に、コマンドウィンドウで、次のコマンドを実行して依存関係をインストールします: pip install pydub

```

C:\Users\14175>pip install pydub
Collecting pydub
  Downloading pydub-0.25.1-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)
Installing collected packages: pydub
Successfully installed pydub-0.25.1

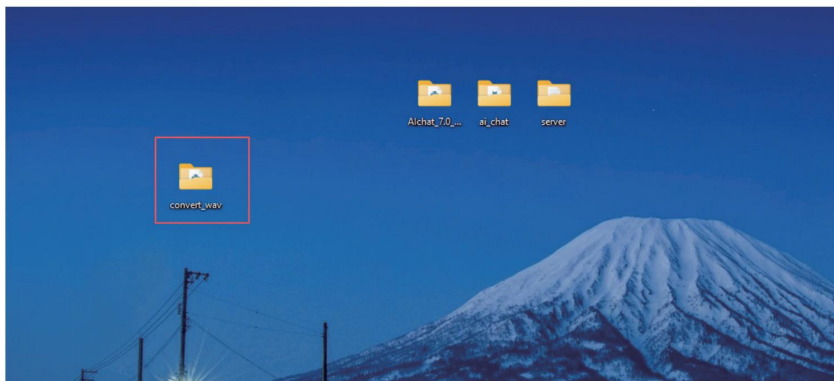
C:\Users\14175>

```

- インストール後、付属のコードパッケージにある、MP3をWAV形式（16kHz、16ビット、ステレオ/デュアルチャンネルの仕様を満たす）に変換するためのスクリプトコードを開いてください。
- 下記のリンクをクリックしてスクリプトコードを開いてください。

https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/convert_wav

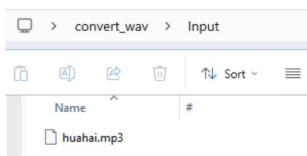
- このスクリプトをデスクトップに配置しました。



- コマンドウィンドウで、このパスに移動します。

```
C:\Users\14175>cd Desktop/
C:\Users\14175\Desktop>cd convert_wav
C:\Users\14175\Desktop\convert_wav>
```

- 次に、MP3ファイルを「Input」フォルダに入れてください。



- このスクリプトコードを実行してください。(Python環境がPython 3.11.2であることを確認してください。)

```
C:\Users\14175\Desktop\convert_wav>python --version
Python 3.11.2
```

- Python 3.13以降:公式チームはaudioopモジュール (pydubが依存しているモジュール)を削除しました。一部のサードパーティライブラリ (pyaudio, pygame, pydubなど)はまだ完全には互換性がありません。
- Python 3.11.xの場合:



安定していて、成熟しており、非常に相性が良い。



音声出力が含まれています。



ほとんどのAI、音声、データ分析ライブラリと完全に互換性があります。

- スクリプトを実行します:

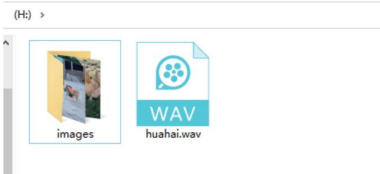
```
C:\Users\admin\Desktop\convert_wav>python mp3_to_wav.py
[OK] huanai.mp3 -> C:\Users\admin\Desktop\convert_wav\Output\huahai.wav (Conversion: Yes)
[ ] Batch conversion completed. All files meet ESP32 I2S requirements.
```

- 生成されたWAVファイルは「Output」フォルダに保存されます。

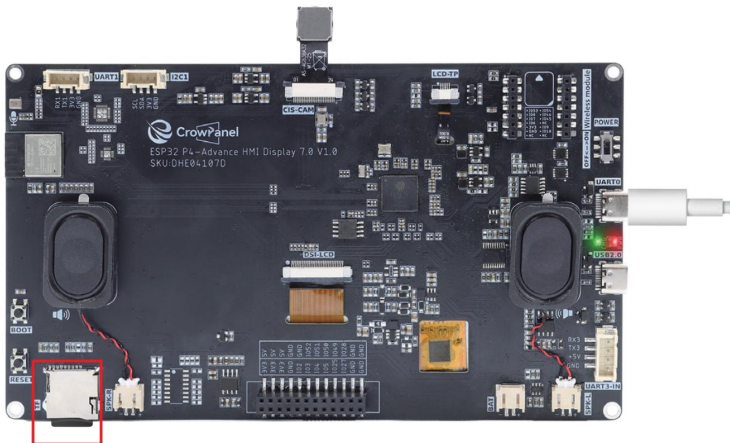
convert_wav > Output



- 次に、このファイルをUSBフラッシュドライブに移動します。



- 最後に、SDカードを取り外し、Advance-P4ボードに挿入します。



主な機能

- メインフォルダはプログラム実行のコアディレクトリであり、メインファイルが含まれています。
関数実行ファイル main.c
- ビルドシステムの「CMakeLists.txt」ファイルにメインフォルダを追加します。

```

EXPLORER
LESSON12
> .vscode
> build
  main
    include
    M CMakeLists.txt
    C main.c
  peripheral
  bsp_audio
    include
    C bsp_audio.h
    C bsp_audio.c
    M CMakeLists.txt
  bsp_sd
    include
    C bsp_sd.h
    C bsp_sd.c
    M CMakeLists.txt
  M CMakeLists.txt
  partitions.csv
  sdkconfig
  sdkconfig.old

OUTLINE
TIMELINE
PROJECT COMPONENTS

main > C main.c > ...
1  /*-----Header file declaration-----
2  #include "main.h"
3  /*-----Header file declaration end-----
4
5  /*-----Variable declaration-----
6  static esp_ldo_channel_handle_t ldo4 = NULL;
7  static esp_ldo_channel_handle_t ldo3 = NULL;
8  /*-----Variable declaration end-----
9
10 /*-----Functional function-----
11 void init_fail(const char *name, esp_err_t err)
12 {
13     static bool state = false;
14     while (1)
15     {
16         if (!state)
17         {
18             MAIN_ERROR("%s init [ %s ]", name, esp_err_to_name(err));
19             state = true;
20         }
21         vTaskDelay(1000 / portTICK_PERIOD_MS);
22     }
23 }
24
25 void Init(void)
26 {
27     static esp_err_t err = ESP_OK;
28     esp_ldo_channel_config_t ldo3_cof = {
29         .chan_id = 3,
30         .voltage_mv = 2500,
31     };
32     err = esp_ldo_acquire_channel(&ldo3_cof, &ldo3);
33     if (err != ESP_OK)
34         init_fail("ldo3", err);
35     esp_ldo_channel_config_t ldo4_cof = {
36         .chan_id = 4,
37         .voltage_mv = 3300,
38     };
39     err = esp_ldo_acquire_channel(&ldo4_cof, &ldo4);
40     if (err != ESP_OK)

```

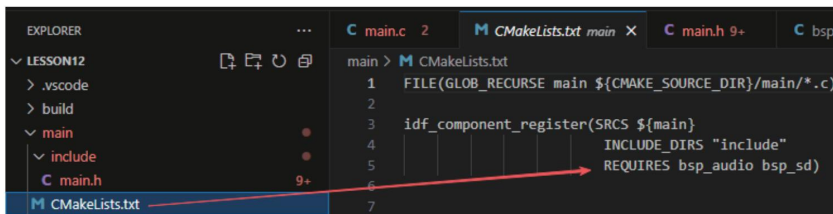
- これはアプリケーション全体のエントリ ファイルです。ESP-IDF には int main() はなく、実行は void app_main(void) から始まります。
- まずはmain.cについて説明しましょう。

熱:

- Init() 関数は、オーディオ再生システムに必要なハードウェアを初期化するために使用されます。これには、LDO3 (2.5V) および LDO4 (3.3V) チャンネルの設定と取得、WAV ファイルを読み取るための SD カードの初期化、オーディオ コントローラの初期化とオーディオ ハードウェアのオフ、および WAV 再生の準備のための I2S オーディオ チャンネルの初期化が含まれます。いずれかのステップが失敗した場合、init_fail() 関数が呼び出され、エラーが出力されてプログラムが停止します。
- 実行。

```
main > C main.c > app_main(void)
25 void Init(void)
28     esp_ldo_channel_config_t ldo3_cof = {
32         err = esp_ldo_acquire_channel(&ldo3_cof, &ldo3);
33         if (err != ESP_OK)
34             init_fail("ldo3", err);
35         esp_ldo_channel_config_t ldo4_cof = {
36             .chan_id = 4,
37             .voltage_mv = 3300,
38         };
39         err = esp_ldo_acquire_channel(&ldo4_cof, &ldo4);
40         if (err != ESP_OK)
41             init_fail("ldo4", err);
42
43         err = sd_init(); /*SD Initialization*/
44         if (err != ESP_OK)
45             init_fail("sd", err);
46         vTaskDelay(500 / portTICK_PERIOD_MS);
47
48         err = audio_ctrl_init(); /*Audio CTRL Initialization*/
49         if (err != ESP_OK)
50             init_fail("audio ctrl", err);
51
52         set_audio_ctrl(false);
53         err = audio_init(); /*Audio Initialization*/
54         if (err != ESP_OK)
55             init_fail("audio", err);
56         vTaskDelay(500 / portTICK_PERIOD_MS);
57     }
58
59 void app_main(void)
60 {
61     MAIN_INFO("-----Demo version-----");
62     MAIN_INFO("-----Start the test-----");
63     Init();
64
65     Audio_play_wav_sd("/sdcard/huahai.wav"); /*Play the WAV file stored on the SD card that
66 }
67 /*-----Functional function end-----*/
```

- SDカードやその他のコンポーネントの初期化が完了するのを待った後、次のステップは、bsp_audioコンポーネントからAudio_play_wav_sdを実行して、SDカードに保存されている変換済みのWAVオーディオファイルを再生することです。
- 最後に、メインディレクトリにある「CMakeLists.txt」ファイルを見てみましょう。
- このCMake設定の役割は以下のとおりです。
 - main/ディレクトリ内のすべての.cソースファイルをコンポーネントのソースファイルとして収集します。
 - メインコンポーネントをESP-IDFビルドシステムに登録し、カスタムコンポーネントbsp_audioおよびbsp_sdへの依存関係を宣言します。
- これにより、ビルドプロセス中にESP-IDFがこれら2つをビルドする必要があることが保証されます。
まず構成要素、次に主要構成要素。



注：以降のコースでは、新しい「CMakeLists.txt」ファイルをゼロから作成することはありません。代わりに、既存のファイルに若干の変更を加えて、他のドライバをメイン関数に統合します。

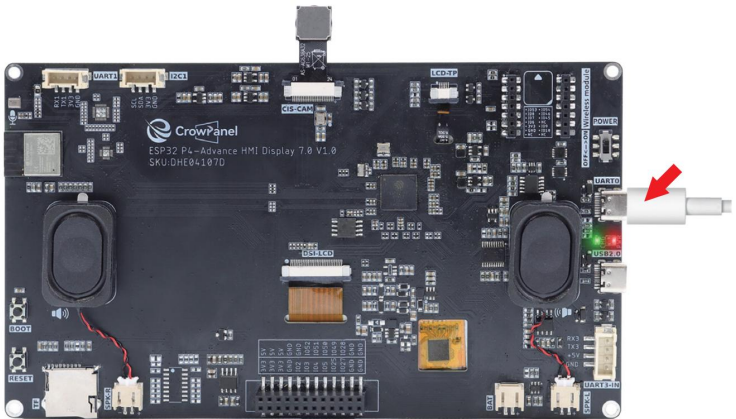
完全なコード

完全なコード実装をご覧になるには、下記のリンクをクリックしてください。

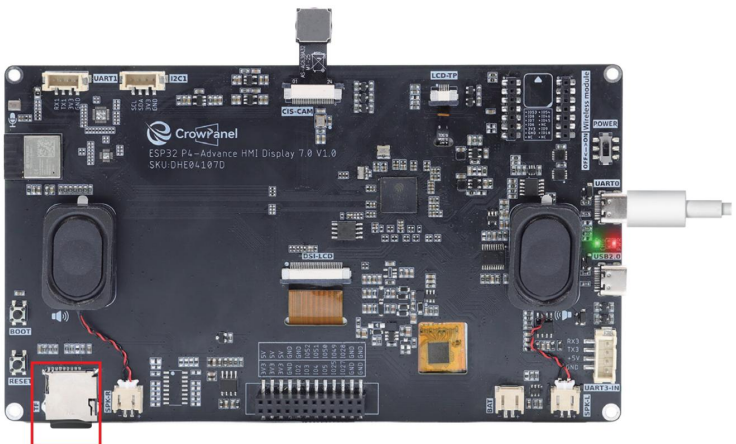
https://github.com/Elecrow-RD/CrowPanel-Advanced-7inch-ESP32-P4-HMI-AI-Display-1024x600-IPS-Touch-Screen/tree/master/example/V1.1/idf-code/Lesson12-Playing_Loca_Music_from_SD_Card

プログラミング手順

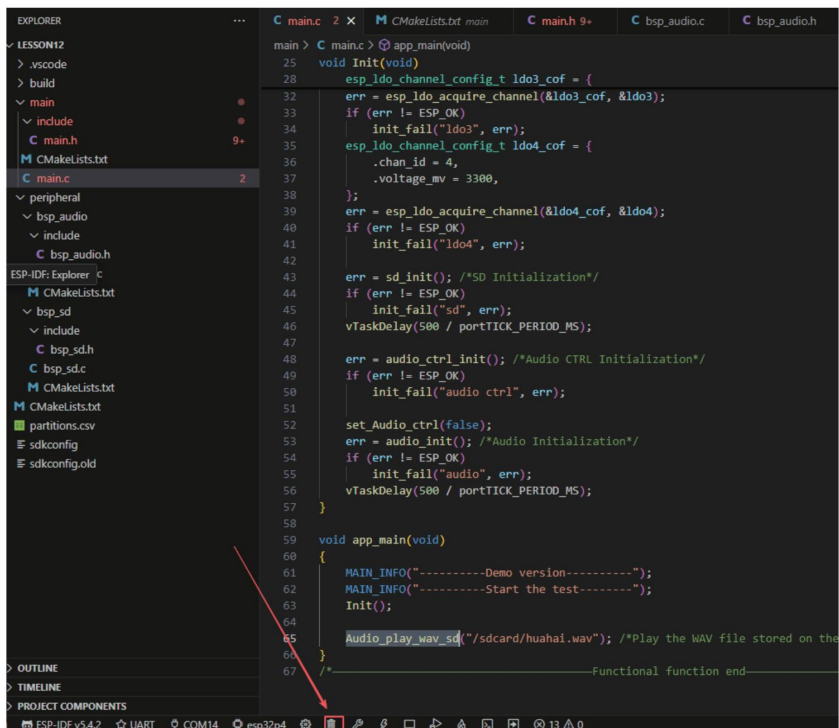
- コードの準備ができたので、次のステップはそれを ESP32-P4 に書き込むことです。
結果を観察してください。
- まず、USBケーブルを使ってAdvance-P4デバイスをコンピュータに接続してください。



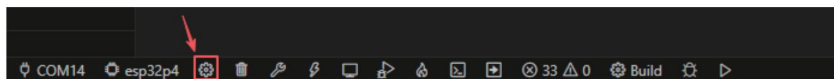
- まず、2つのことを再確認してください。変換されたWAVオーディオファイルがSDカードに保存されているか、そしてSDカードがSDカードスロットに挿入されているかです。
アドバンスP4。



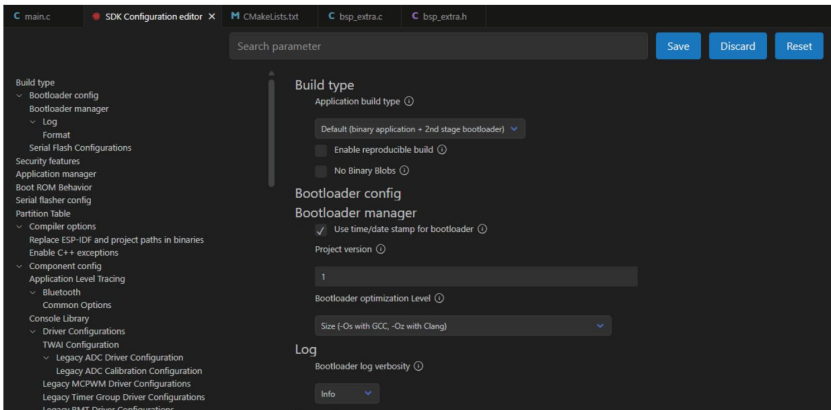
- ・フラッシュ書き込みの準備を開始する前に、コンパイルによって生成されたすべてのファイルを削除して、プロジェクトを初期の「未ビルド」状態に戻してください。これにより、以降のコンパイルが以前の操作の影響を受けないことが保証されます。



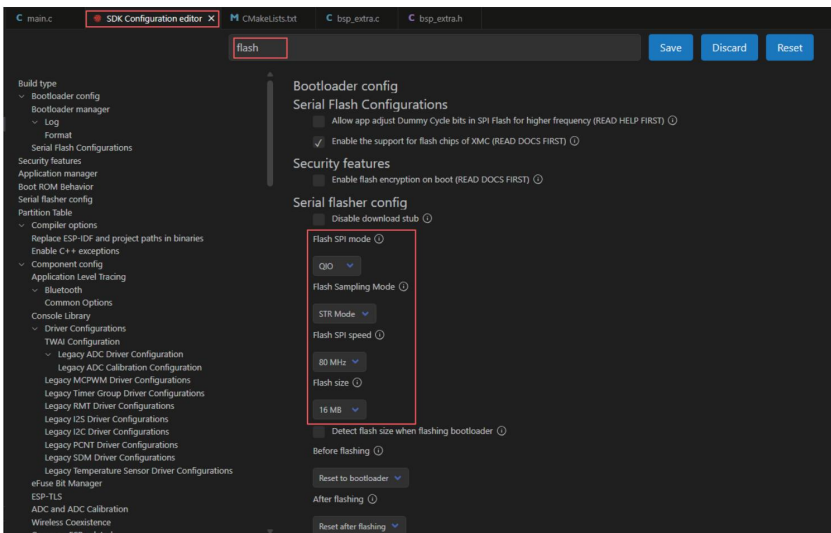
- ・まず、最初のセクションの手順に従って、ESP-IDFバージョンを選択し、コードをアップロードします。
方法、シリアルポート番号、および対象チップ。
- ・次に、SDKを設定する必要があります。
- ・下の図のアイコンをクリックしてください。



- しばらく読み込みが続くと、関連するSDKに進むことができます。
構成。



- このプロジェクトでは画面に情報を表示する必要があるため、まずレッスン7のSDK構成を参照して画面表示設定を完了し、画面が正しく機能することを確認することをお勧めします（詳細は101～103ページを参照してください）。
- 次に、検索ボックスに「flash」と入力します。（フラッシュの設定が一致していることを確認してください。）
私の。）

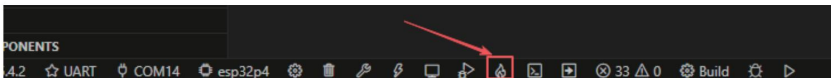


- 設定が完了したら、必ず設定を保存してください。

- 次に、コードをコンパイルしてフラッシュします（最初のレッスンで詳しく説明します）。

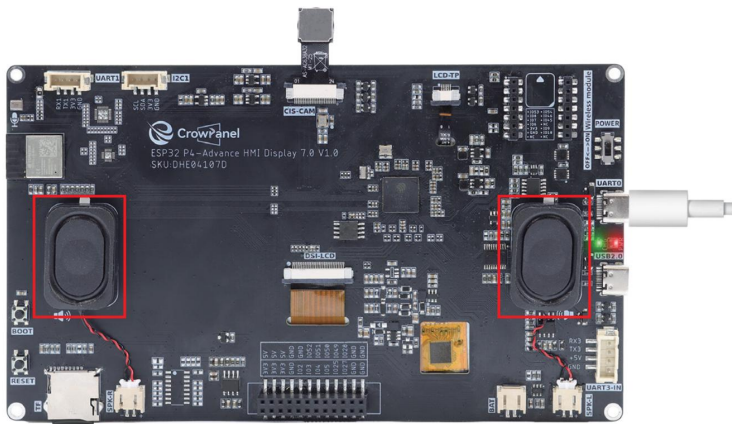
- ここで、非常に便利な機能をご紹介します。ボタン1つで実行できます

コンパイル、アップロード、および監視起動を一度に実行します（ただし、コード全体にエラーがないことが前提です）。



- しばらく待つと、コードのコンパイルとアップロードが完了し、モニターが自動的に開きます。

- コードが実行されると、Advance-P4 のスピーカーからWAV ファイルが再生されます。
音声データはSDカードに保存されています。



SDカードに保存されているWAYの音声ファイルが再生されました。